Author: Kavindu Hasaranga (IoT Trainee)
Date: 21st October 2026 | Week 4

System Architecture Specification

CODEXUS STOCKSENSE | AI-POWERED SMART INVENTORY SYSTEM

1. Executive System Overview

StockSense is an offline-first, edge-connected inventory management platform engineered for high-density physical tracking in resource-constrained industrial environments. The system architecture coordinates continuous 24-bit weight telemetry, RFID item identification, climate sensing, and AI-driven forecasting across five interconnected execution layers: *Sense*, *Edge Processing*, *Connect Transport*, *Cloud Infrastructure*, and *Insight Analytics*.

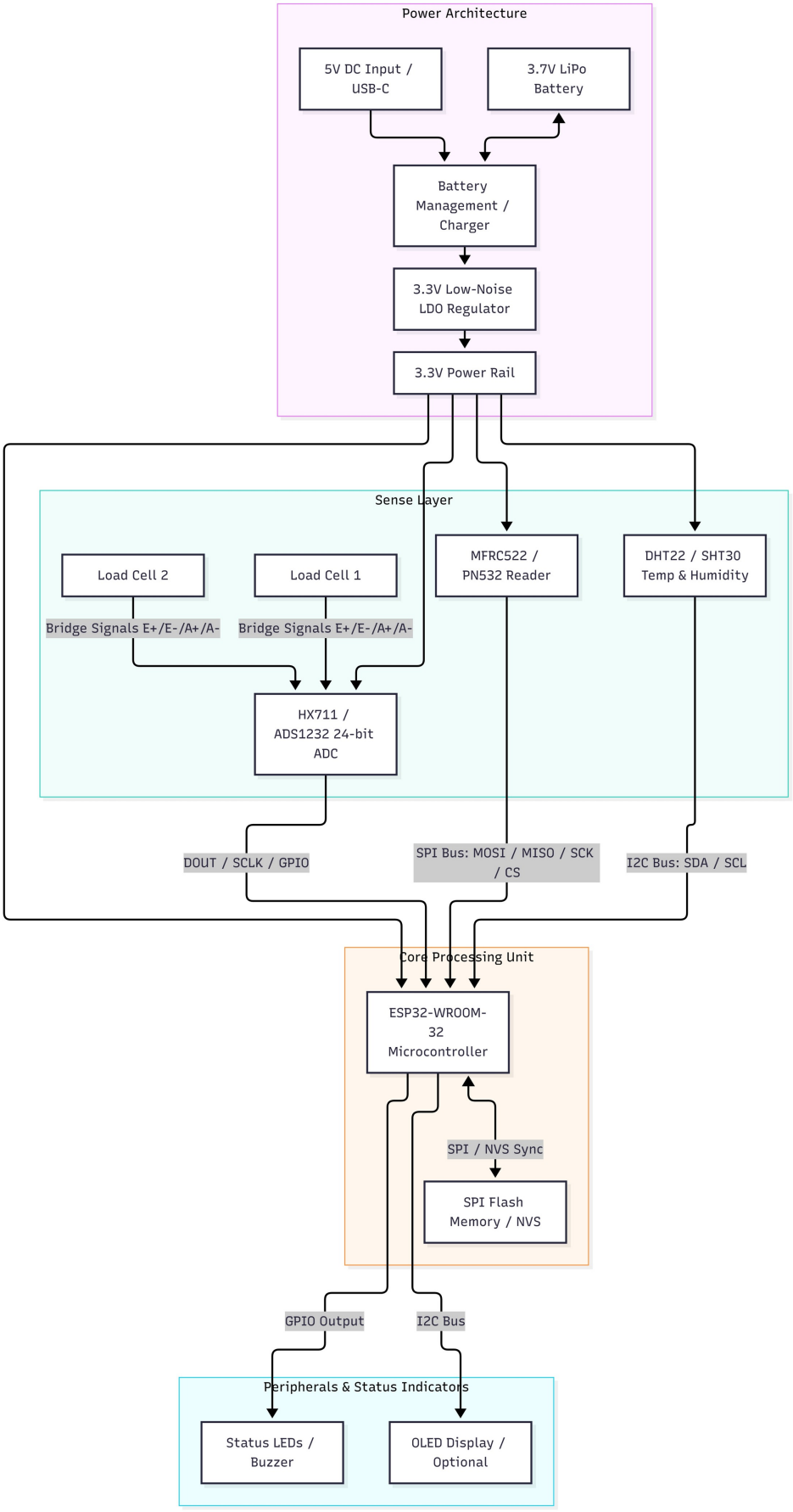
2. Hardware Subsystem Architecture

The edge hardware centers on an ESP32 micro-controller paired with a precision analog front-end (AFE), power management module, and local storage. The platform features an integrated battery management circuit enabling seamless failover during grid outages. The full block diagram is provided on the following page (Figure 1).

Subsystem Module	Primary Component / IC	Interface / Protocol	Operational Role
Core MCU	ESP32-WROOM-32 (Xtensa 32-bit)	Internal Bus / FreeRTOS	Edge logic, filtering, Wi-Fi/MQTT stack
Analog Front-End	HX711 24-bit Delta-Sigma ADC	2-Wire Serial (DOUT/PD_SCK)	Amplification (Gain 128) & digitizing strain gauge
Item Scanner	MFRC522 / PN532 RFID Module	SPI Bus (10 MHz)	Reads bin SKU tags & personnel access cards
Power Manager	TP4056 / Ideal Diode Power-Path	5V VBUS / 3.7V LiPo	Continuous rail regulation & power cut switchover

StockSense Hardware Architecture Block Diagram

Figure 1: Hardware Subsystem Block Diagram (Power, Sense, Core MCU, and Peripherals)



3. Edge Firmware Architecture

The ESP32 firmware is structured on **FreeRTOS** to execute concurrent real-time sensing, signal processing, and communication tasks. It utilizes an event-driven Finite State Machine (FSM) to calculate item quantities, manage tare calibration, and dispatch telemetry payloads. The task and execution pipeline is shown in Figure 2.

Real-Time Digital Signal Processing

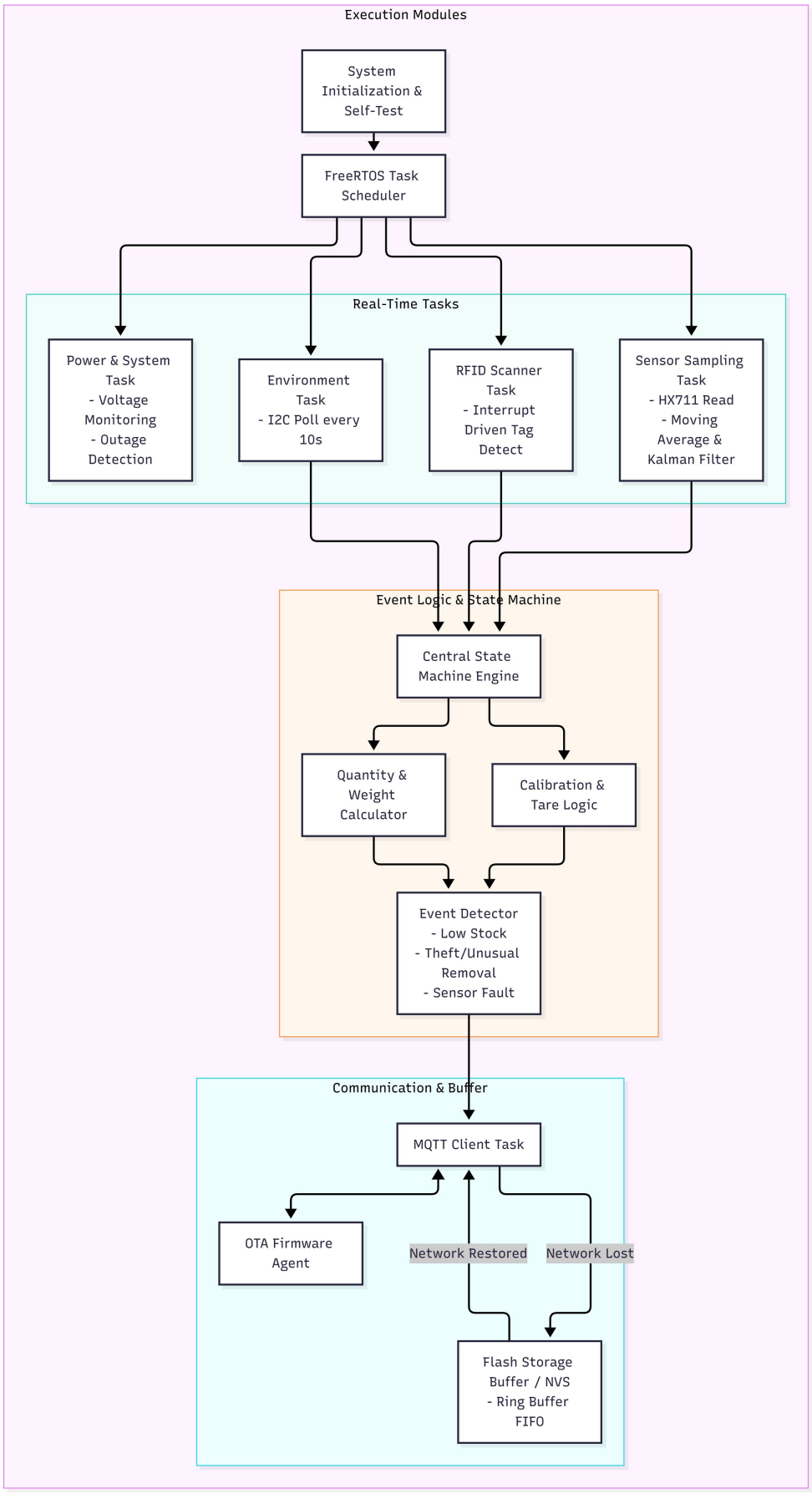
Raw 24-bit strain gauge counts undergo a two-tier filtering pipeline: a 5-sample median filter to eliminate mechanical impulse spikes, followed by an exponential moving average (EMA) or simple Kalman filter to dampen ambient floor vibration before computing quantity updates.

Offline-First Ring Buffer (SPIFFS)

During Wi-Fi network or server disconnections, serialized JSON telemetry logs are written to an SPIFFS/NVS non-volatile flash ring buffer (up to 10,000 records). Upon MQTT re-connection, an automatic sync agent flushes queued historical records sequentially without data loss.

StockSense Firmware Architecture Diagram

Figure 2: FreeRTOS Firmware Task & Execution Pipeline Diagram



4. End-to-End Data Flow Sequence

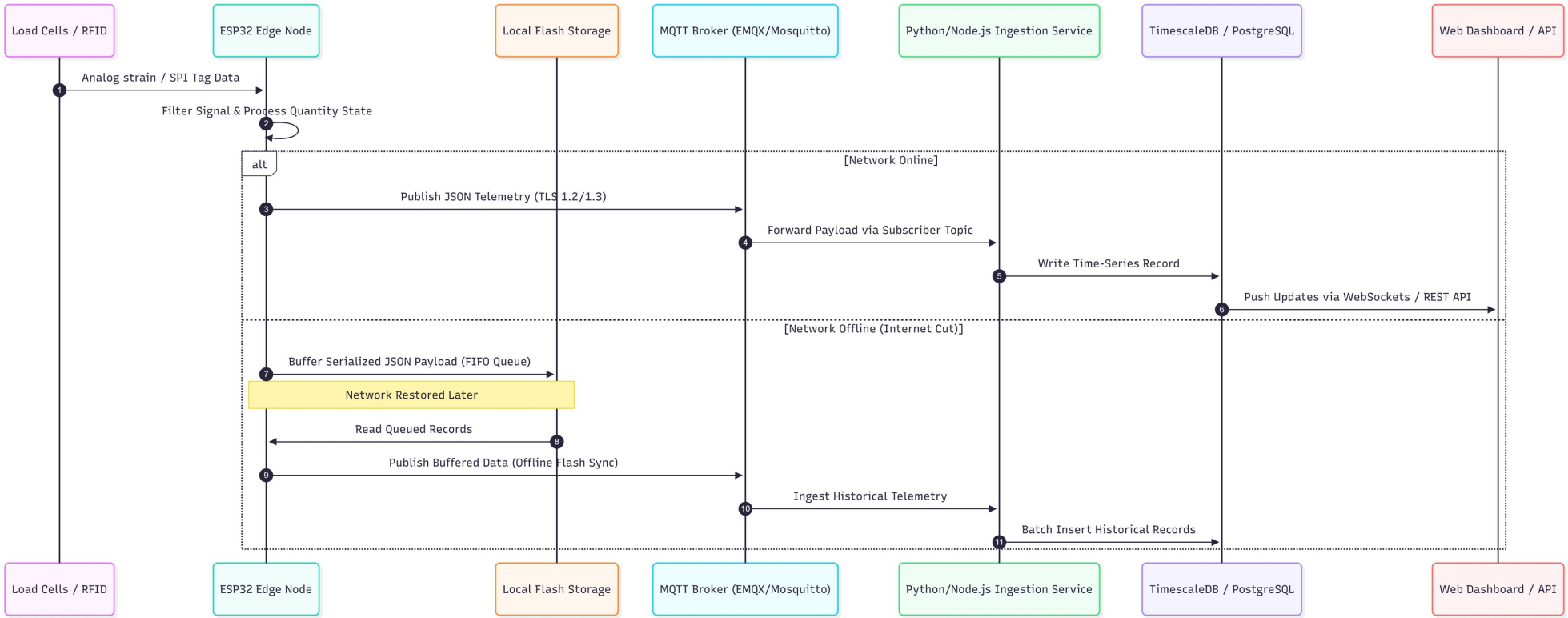
Data propagates left-to-right from physical load sensing platforms through edge processing and secure transport to cloud time-series storage and real-time dashboard subscriptions. The sequence diagram (Figure 3) maps operational handling across both normal online conditions and network outage recovery states.

Data Pipeline Lifecycle Phases

- **Edge Processing:** Analog bridge signals sampled at 10 Hz; local state derived within 100 ms. If delta weight exceeds threshold (e.g., > 5g step change), a stock movement transaction is formed.
- **Transport Dispatch:** Payload published via MQTTS (Port 8883) with QoS 1 to guarantee delivery.
- **Ingestion & Storage:** Python/Node.js ingestion worker validates JSON schema and performs multi-row inserts into TimescaleDB hyper-tables indexed by site, bin, and timestamp.
- **Dashboard Synchronization:** WebSockets push live inventory state updates to active React client sessions in under 2.0 seconds end-to-end.

StockSense Data Flow Sequence Diagram

Figure 3: Sequence Diagram for Online Ingestion & Offline Flash Resynchronization



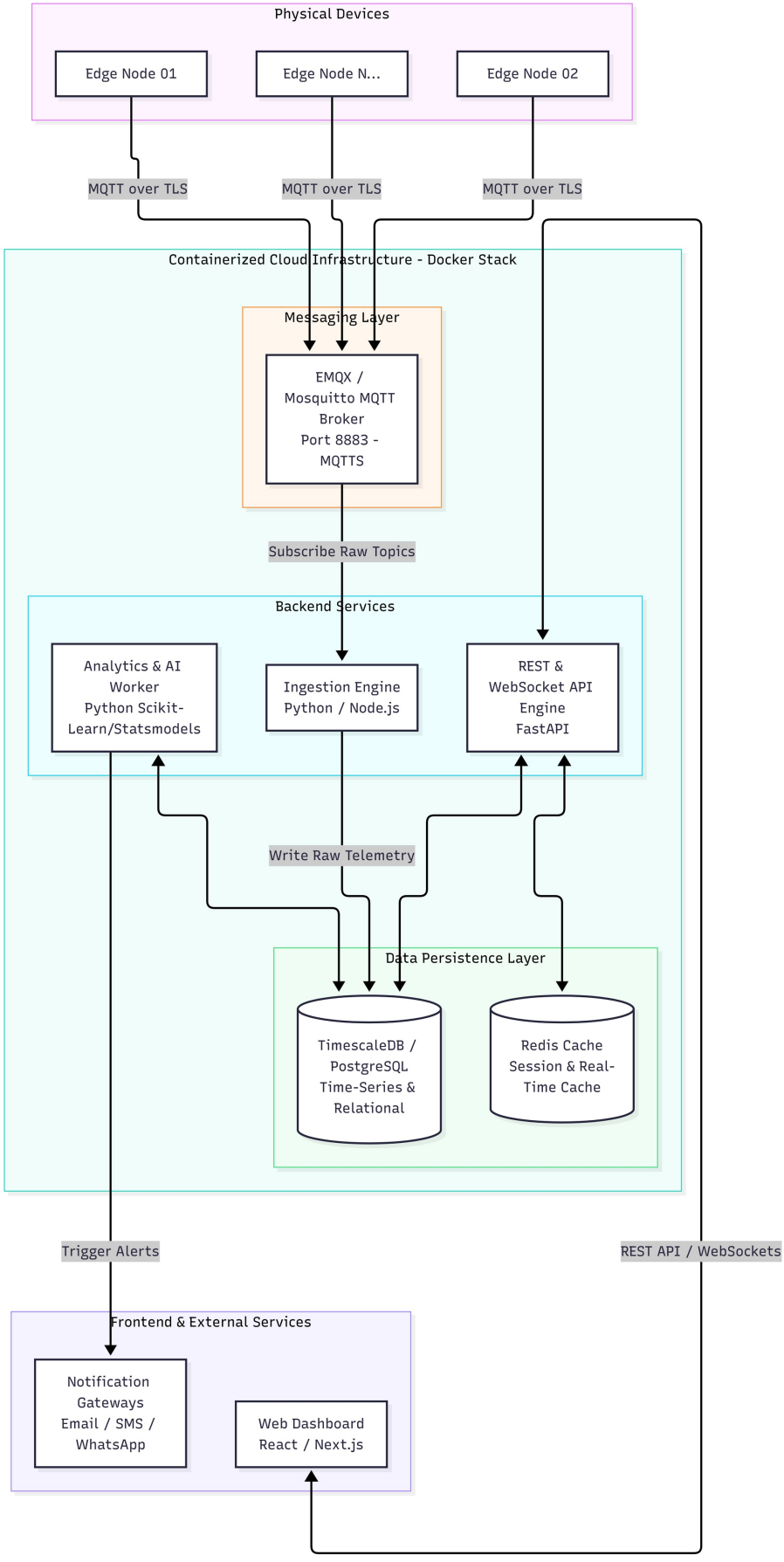
5. Cloud Infrastructure Platform

The cloud backend relies on a containerized **Docker Stack** deployed on high-availability cloud VPS infrastructure. It segregates messaging brokers, ingestion pipelines, REST/WebSocket API services, time-series storage, and automated machine learning workers. The architecture is shown in Figure 4.

EMQX / Mosquitto MQTT Broker Handles MQTTS connections over TLS 1.2/1.3. Manages topic hierarchies (`stocksense/site_id/bin_id/telemetry`), client credential authentication, and event routing to internal message queues.	**TimescaleDB & Analytics Stack** Extends PostgreSQL with automated time-series partitioning. An isolated Python analytics worker runs Holt-Winters exponential smoothing to forecast stock run-out dates and trigger low-stock alerts.

StockSense Cloud Platform Containerized Architecture

Figure 4: Docker Containerized Infrastructure & Backend Microservices Stack

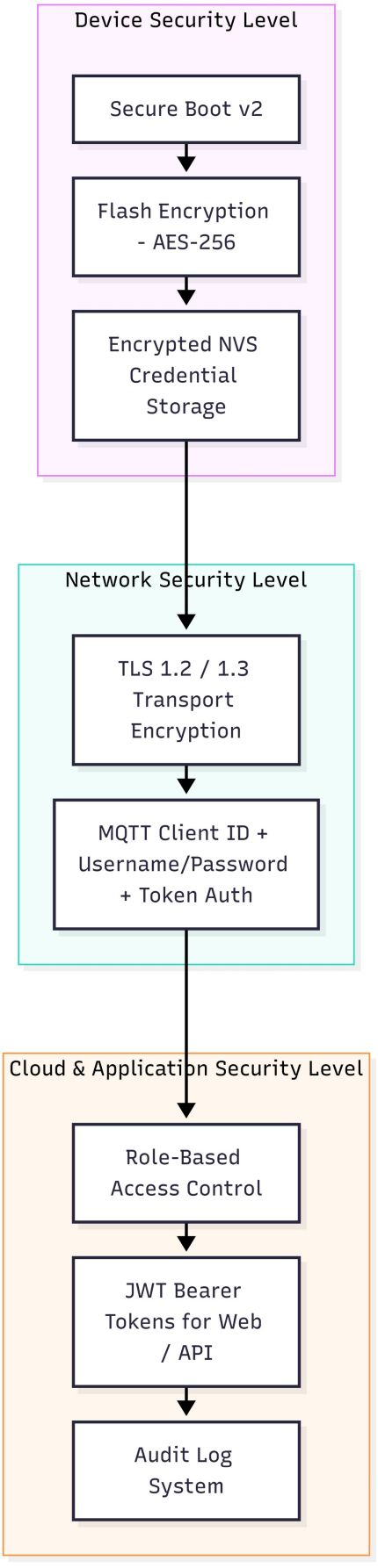


6. Multi-Layer Security Architecture

Security is enforced across all system layers—protecting hardware flash memory against extraction, encrypting network transport, and isolating multi-tenant enterprise data in cloud services. The security stack is shown in Figure 5.

StockSense Multi-Layer Security Architecture Diagram

Figure 5: Security Stack across Device, Network Transport, and Application Layers



7. Architecture Verification Summary

Security & Performance Layer	Implementation Strategy	Target KPI / Metric
Device Firmware Integrity	ESP32 Secure Boot v2 + AES-256 Flash Encryption	Zero unauthorized binary execution
Transport Security	MQTT over TLS 1.2/1.3 with X.509 token auth	100% encrypted telemetry in transit
Cloud Multi-Tenancy	RBAC with OAuth2 / JWT bearer authorization tokens	Strict tenant data isolation
Offline Resilience	Local SPIFFS FIFO Queue & Auto Sync Engine	≥ 10,000 logs stored without data loss